ECON 370: Assignment 1

Jiaxi Li

Due: Before class on September 11th, 2025

Names of all group members:

Directions

As a reminder, LLMs and AIs are strictly forbidden from use on this assignment.

Use the Rmd template and write code to answer the following questions. In order to ensure smooth grading, I will ask you to write both your code and explanation (as comment) in code chunk. Please write your code and answer in the corresponding place.

Each question is worth three points. For each section, a "question" corresponds to a number. So each bullet point in "Reading Documentation" section will be worth one point.

Please comment your code clearly. 0.5 point per question will be deducted if there is a significantly lack of commenting. Also, the easier your code is to read, the more likely I will be able to figure out what you're doing.

You may work in groups of three to four people for this homework. Please submit one script per group and clearly indicate who was in your group for the assignment.

I. Reading Documentation

In this section, I will be asking you to read the documentation for functions that we have yet to cover or use. Remember, if the name of a function is "fun1", you can pull up its documentation by running either "help(fun1)" or "?fun1" in the R Console. Also, when answering what each function does, *please do not just copy and paste the description of the function in the documentation.* Please respond inside the code chunk as comments, using your own words.

Note: Do not print the function documentation in Rmd, but rather run it in R console.

1. Pull up the documentation for the "log" function and answer the following questions:

- Describe what this function does.

- What are its arguments and what type should each argument be?

- What is the default base for the logarithm? Hint: Remember that the natural log is "log base e." Does there appear to be a difference between the "log" and "logb" functions? What does the "log10" function do? Please answer each part as they are all related.

2. Pull up the documentation for the "rnorm" function and answer the following questions.
   Note: This is a little more challenging than the above question as it pulls up the documentation for a *family* of functions and not just the "rnorm" function, so please keep this in mind.

- What does the "rnorm" function specifically do? Read the description in the documentation very carefully and match it with the "usage" section.

- What are the arguments for the "rnorm" function along with their types? Are there any default arguments? What are they?

- Write the code to draw 100 values from a normal distribution with mean 5 and *variance* of 4. Note: remember that the variance is the standard deviation squared.

3. Pull up the documentation for the "optim" function. This will be the most challenging question. Please answer the following:

- What does the "optim" function do? (Hint: "optimization" means to maximize or minimize some function.)

- Which arguments *must* be supplied to the "optim" function?

- The "control" argument is a list of options that control the algorithm used for the specific optimization method. What is the default maximum number of iterations? (There will be three different values depending on the "method".) What is the typical value of "reltol"? (Hint: Follow the instructions in the "control" argument.)

II. Creating Objects

1. Create a vector of you and your group members' names called "group_names".

2. Create a vector of the integers from 1 to 5 called "my_vec".

3. Update "my_vec" by adding 10 to each element.

   - Hint: make use of vectorized operations.
   - Note that you should still call this new vector "my_vec".

4. Create a vector of integers from 1995 to the current year called "years_alive".

5. Create a vector of logicals testing if an element in "years_alive" is a leap year.

6. Draw 1000 simulations from a "N(0,1)" distribution and call it "norm_draws".

   - Hint: use "rnorm".

7. Create a vector that is the log of the draws from "norm_draws", called "log_draws".

   - Note: this will produce NaNs and that's okay.

8. Create a logical vector called "draws_NaN" that tests which elements of "log_draws" are NaNs.

9. Create a vector of integers from 1 to "N", where "N" is the number of NaN elements in "log_draws".

10. Use modular arithmetic operations to create a vector called "class_length" whose first element is the number of whole hours in one of our class meetings and the second is the number of leftover minutes. E.g., 75 would be "c(0,75)".

    - Note: Our class meetings are 75 minutes long.

III. Creating Data Sets

1. Create a variable called "mtcars_list" that stores each variable in "mtcars" as a different element in the list.

    - Hint: use "as.list()"

2. Create a "2 x 13" matrix of the letters of the alphabet, appearing left to right. Call this matrix "letter_mat".

3. Create a "5 x 5" matrix of new "N(0,1)" draws called "norm_draws_mat".

    - Hint: How many draws will we need for a "5 x 5" matrix?

4. Using the "iris" dataset, store the "Species" variable as a factor variable named "iris_species", where "versicolor" is the first level, "virginica" is the second, and "setosa" is the third.

5. Create a sequence of numbers from 1.2 to 5.3 by 0.05 called "first_seq".

6. Create a sequence of numbers from 1.2 to 5.3 that is 100 elements long called "second_seq".

7. Load the AER library and attach the "GSOEP9402" dataset ("data("GSOEP9402")"). What are the dimensions of the dataset? What are the names of the variables? Rename the variables to the same names, but all in uppercase.

8. Create the following vectors:

    - "person_id": numeric vector from 1 to 50, increasing by 1.
    - "time": numeric vector from 2001 to 2020, increasing by 1.

9. Create a "data.frame" called "panel_data" with the following variables:

    - "id": each element of "person_id" repeated 20 times.
    - "time": the "time" vector repeated 50 times.
    - "draw": the vector "norm_draws".
    - Note: this dataset is just for practice; do not copy/paste an object 20/50 times.

IV. Indexing

1. Store a vector of the ids of the elements in "draws_NaN" that are NaN called "NaN_ids".

- Note: use the "which()" function with "draws_NaN".

2. Store a vector of the ids of the elements in "norm_draws" that are positive called "pos_ids".

   - Note: positive means *strictly* greater than 0 (does not include 0).

3. Store the elements with even indices from "norm_draws" called "even_id_draws".

   - Hint: use the "seq()" function.
   - Style Tip: do not hard code in 1000; use "length()" instead.

4. Extract the value from the 20th observation of the "mpg" variable in "mtcars" using three different ways.

5. Extract the values from the 17th observation of the "cyl" and "hp" variables in "mtcars".

6. (Harder) Extract all observations from "mtcars" that have 8 cylinders ("cyl") using logical expressions/indexing.

7. Load the "Titanic" dataset (note: it is "Titanic", not "titanic").

   - Use "str()" to determine the dimensions and structure of the dataset.
   - Print outcomes for the following subgroups:
     1. Female, adult, crew
     2. Male, first-class, adults
     3. Female, first-class, children
     4. Male, second-class, adults

8. Using "mtcars_list", extract the "wt" variable *as an atomic vector* using two different list indexing methods.

9. Using "mtcars_list", extract the "wt" variable *as a list*.